



# Structural Hierarchy Extraction Engine

**Automated Feature Hierarchy Extraction from Any Product Documentation URL**

From any help center or product page → Structured 3-layer intelligence in seconds.

# The Problem We Solved

Product Managers and competitive analysts spend hours manually reading competitor help centers, product pages, and documentation sites — then trying to mentally map out what features those products have. This process is slow, inconsistent, and does not scale across multiple competitors.

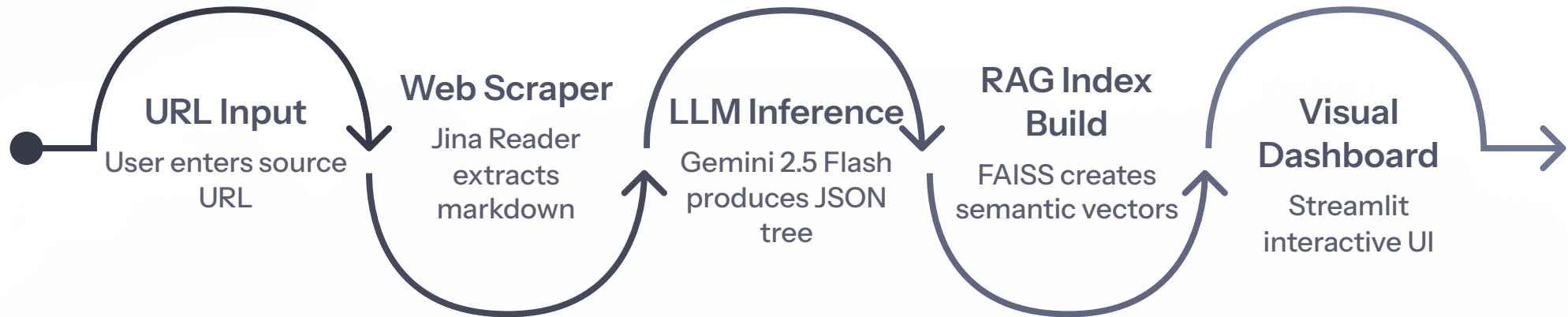
## The Core Challenge

Given any company's documentation URL, can we automatically extract a clean, structured map of what the product does — organised into Major Modules, Features, and Sub-features — without any manual reading?

## Universal Adaptability

This solution needed to work seamlessly across wildly different site types: Notion-style knowledge bases, Zendesk help centres, Intercom docs, custom developer documentation, and product marketing pages — each with different structure, depth, and content density.

# Our Solution Architecture



The end-to-end pipeline automatically ingests raw documentation, processes it through advanced language models, builds a local semantic index, and serves it within a modern visual workspace.

# Why Three Separate Files?

A highly modular design decouples ingestion, extraction, and presentation to ensure robust debugging and scaling.

## **scraper.py**

### **Extraction Layer**

Serves as a standalone proof-of-concept. Validates that any URL can be converted to clean markdown text using the Jina Reader API before wiring it into the full pipeline.

## **generator.py**

### **Inference Layer**

Acts as a standalone LLM interface. Takes scraped markdown and prompts Gemini 2.5 Flash to produce a structured JSON hierarchy, allowing rapid prompt tuning.

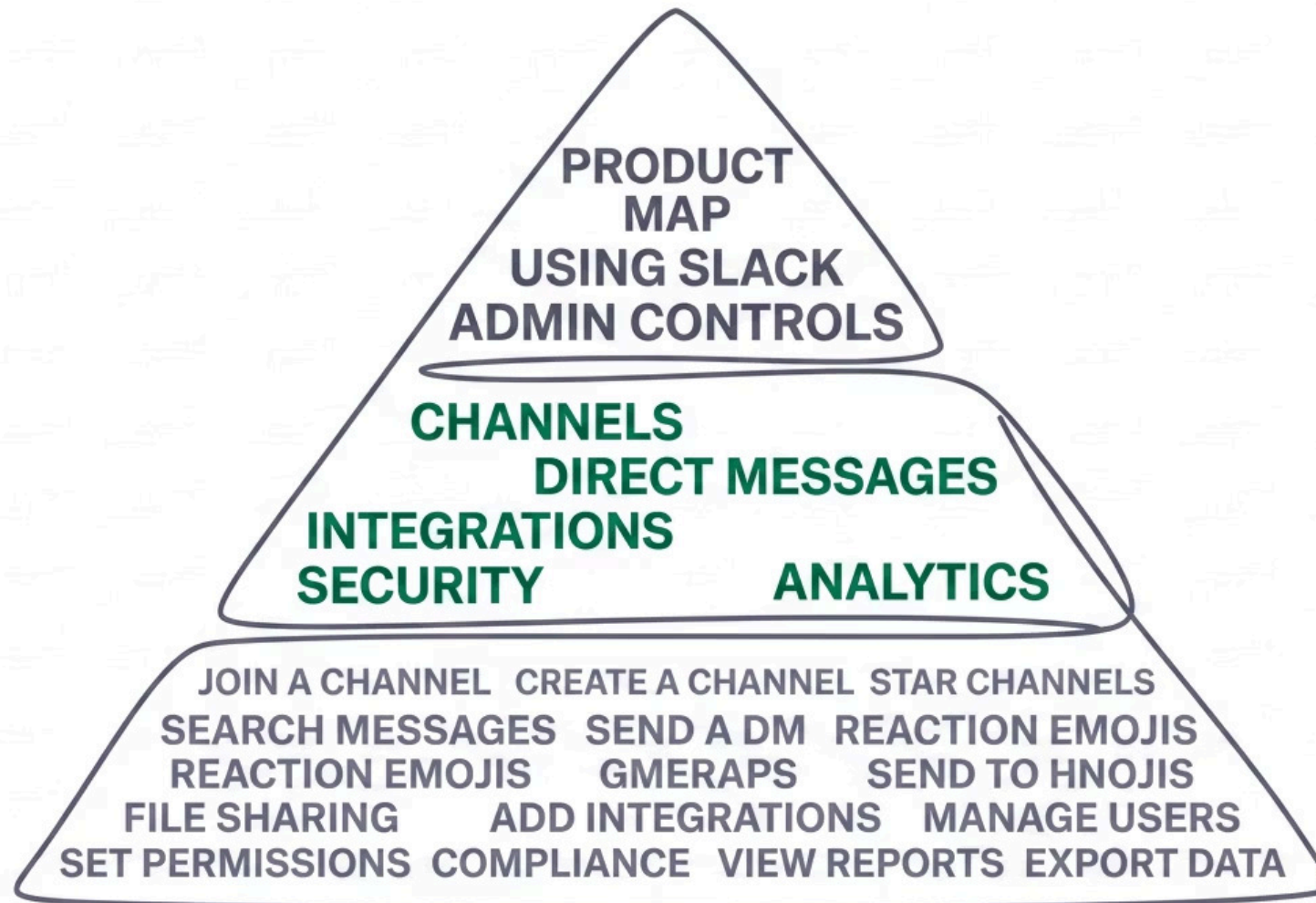
## **app.py**

### **Production Application**

The full-stack coordinator. Combines scraping, LLM inference, RAG indexing, and the interactive Streamlit user interface into a single unified deployment.



# The Output: 3-Layer Feature Hierarchy



The LLM dynamically decides the depth based on content density. Shallow landing pages get 2 layers, whereas dense API docs may get up to 4 layers. The system never forces a rigid structure; it reads and shapes the tree organically.

# Dashboard Metrics Explained

|   |                                                                                                                                                                                     |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>System Ingestion</b><br>Tracks how many pages were crawled from the target URL and how many total characters were extracted. Tells the analyst the raw volume processed.         |
| 2 | <b>Processing Logic</b><br>Confirms the inference engine used was the dynamic LLM-inferred model and displays the estimated token load sent to the model.                           |
| 3 | <b>Quality Assurance</b><br>Confirms the output JSON successfully passed structural schema contract validation, ensuring it can be rendered without errors.                         |
| 4 | <b>Top-Level Modules</b><br>The count of Layer 1 root nodes found. A high number indicates a broad, complex product, while a low number points to a highly focused tool.            |
| 5 | <b>Total Structured Nodes</b><br>The sum of all modules, features, and sub-features combined. This serves as a reliable proxy for the total feature surface area of the competitor. |
| 6 | <b>Inferred Hierarchy Depth</b><br>The maximum nesting depth found. Standard products sit at 3, simple landing pages at 2, and highly technical developer portals may reach 4 or 5. |

# Conversational Search Terminal (RAG)

After the feature hierarchy is extracted, analysts often need to ask specific questions about competitor capabilities rather than manually scanning the whole tree. Our integrated Retrieval-Augmented Generation (RAG) terminal solves this instantly.

## Vector Index Build Process

The scraped text is parsed into overlapping 1,000-character chunks using LangChain's RecursiveCharacterTextSplitter, embedded via Google's text-embedding-004, and stored in a local FAISS index.

## Semantic Pinpoint Retrieval

When an analyst submits a query, the top 3 semantically closest documentation chunks are pulled. Gemini 2.5 Flash answers exclusively based on this context, completely preventing hallucinations.

# Guardrails Built Into the System

We implemented multiple production-grade guardrails to ensure system security, stability, and high data quality.



## URL Security Validation

Checks inputs against a private IP blocklist to prevent Server-Side Request Forgery (SSRF) attacks on internal network resources.



## Input Length Capping

Caps text sent to the LLM at 250,000 characters to prevent context window overflow and contain API consumption costs.



## JSON Fence Stripping

Automatically cleans markdown code wrappers (e.g., ``json) from raw model outputs before attempting parser execution.



## Regex Label Cleaning

Defensively strips brackets like "[LAYER 1]" from output names prior to UI rendering to keep the interface clean.



## RAG Score Threshold

Discards FAISS similarity matches scoring above 1.45 to guarantee only closely relevant text blocks influence the answer.



# Evals: How We Measured Quality

We treat evaluation as a core architectural layer, applying quantitative checks to unstructured AI outputs to ensure complete reliability.

| Layer            | What We Evaluated                                      | How We Checked It                                   |
|------------------|--------------------------------------------------------|-----------------------------------------------------|
| Scraping Layer   | Text extraction completeness from raw URL              | Flags failures if total characters are under 100    |
| LLM Output       | Valid JSON format and minimum logical branches         | Schema validation with key fallback checks          |
| Structural Depth | Sensible categorization depth (2 to 5 levels)          | Recursive node tree depth verification              |
| RAG Retrieval    | Direct relevance of retrieved documentation chunks     | Strict FAISS similarity threshold filter (1.45)     |
| RAG Answers      | Strict adherence to source context, zero hallucination | Grounded prompt engineering & low temperature (0.1) |

# Summary and Key Takeaways

## 1. End-to-End Automation

Transform complex documentation URLs into clean, structured 3-layer feature hierarchies automatically, eliminating manual spreadsheets.

## 2. Dynamic Adaptability

The engine shapes itself to the source material, ensuring high accuracy across Notion, Zendesk, Intercom, or custom technical docs.

## 3. Production-Ready Guardrails

Built-in security sanitisation, schema validation, and context boundaries prevent common vulnerabilities and hallucinated data.

## 4. RAG-Powered Insights

Includes a semantic chatbot that answers targeted competitor feature questions using verified documentation as its sole source of truth.

*Built with Gemini 2.5 Flash · text-embedding-004 · FAISS · LangChain ·  
Streamlit · Jina Reader API*

